COS 344: L16 Chapter 11: Texture Mapping

Cobus Redelinghuys

University of Pretoria

16/04/2026

- Today, we will finish our discussion on texture mapping.
- Reminder to book for Practical 4 demos!
- Reminder to form teams for the homework assignment!
- If you are struggling to make a team, let me know.

Introduction

- On Monday we discussed the idea of texture mapping and the required "machinery" to look up texture coordinates.
- In this section we will discuss some of the important techniques in texture mappings.

Section 11.4.1: Controlling Shading Parameters

- The most basic use of texture maps is to look up diffuse colors for shading calculations.
- Apart from diffuse colors what else can we look up?

Section 11.4.1: Controlling Shading Parameters

- The most basic use of texture maps is to look up diffuse colors for shading calculations.
- Apart from diffuse colors what else can we look up?
  - Specular reflectance
  - Specular roughness
- Examples?

Figure 11.20. A ceramic mug with specular roughness controlled by an inverted copy of the diffuse color texture.

Section 11.4.2: Normal Maps and Bump Maps

- In Section 9.2 we saw that the surface normal does not need to be the same as the geometric normal.
- We can leverage this fact to calculate the shading.
- Normal mapping takes advantage of this by storing the surface normals in a texture.
- Three values are stored in each texel which is then interpreted as normals instead of colors.

- The simplest way to store normals, is to store them in the same coordinate system that is used to store the object.
- We can thus just read the normal and use it without any extra work.
- But what is a possible problem?

- The simplest way to store normals, is to store them in the same coordinate system that is used to store the object.
- We can thus just read the normal and use it without any extra work.
- But what is a possible problem?
- What if the geometric object moves?
  - Is the normal still the same?

- The simplest way to store normals, is to store them in the same coordinate system that is used to store the object.
- We can thus just read the normal and use it without any extra work.
- But what is a possible problem?
- What if the geometric object moves?
  - Is the normal still the same?
- Solution:
  - Define the normal in a coordinate system that is attached to the surface.
  - Such a coordinate system can be defined based on the tangent space of the surface.
    - Section 2.7

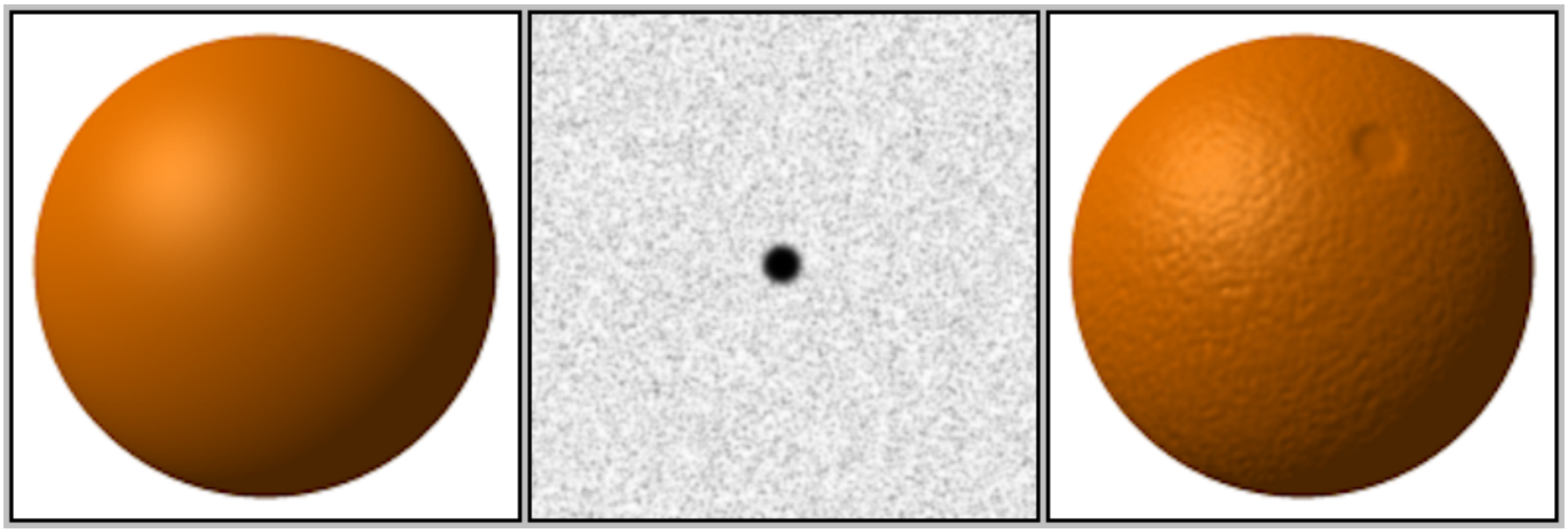
- Where do normal maps come from?

- Where do normal maps come from?
  - Computed from more complex models to which smooth surfaces are approximations.
  - Alternatively they can be measured directly from real surfaces.
  - Alternatively, an alternative is create them as part of the modeling process.
    - For this case it is often nice to use a bump map to specify the normals.
- What are bump maps?

- Where do normal maps come from?
  - Computed from more complex models to which smooth surfaces are approximations.
  - Alternatively they can be measured directly from real surfaces.
  - Alternatively, an alternative is create them as part of the modeling process.
    - For this case it is often nice to use a bump map to specify the normals.
- What are bump maps?
  - Bump maps is a height field.
- What are height fields?

- Where do normal maps come from?
  - Computed from more complex models to which smooth surfaces are approximations.
  - Alternatively they can be measured directly from real surfaces.
  - Alternatively, an alternative is create them as part of the modeling process.
    - For this case it is often nice to use a bump map to specify the normals.
- What are bump maps?
  - Bump maps is a height field.
- What are height fields?
  - A function that gives the local height of the detailed surface above the smooth surface.
- Thus in a bump map if:
  - High values (bright areas of the map if displayed as an image)
    - The surface is protruding outside the smooth surface
  - Low values (darker areas of the map if displayed as an image)
    - The surface is receding below the smooth surface.

Figure 11.21. A wood floor rendered using texture maps to control the shading. (a) Only the diffuse color is modulated by a texture map. (b) The specular roughness is also modulated by a second texture



https://upload.wikimedia.org/wikipedia/commons/0/0a/Bump-map-demo-full.png

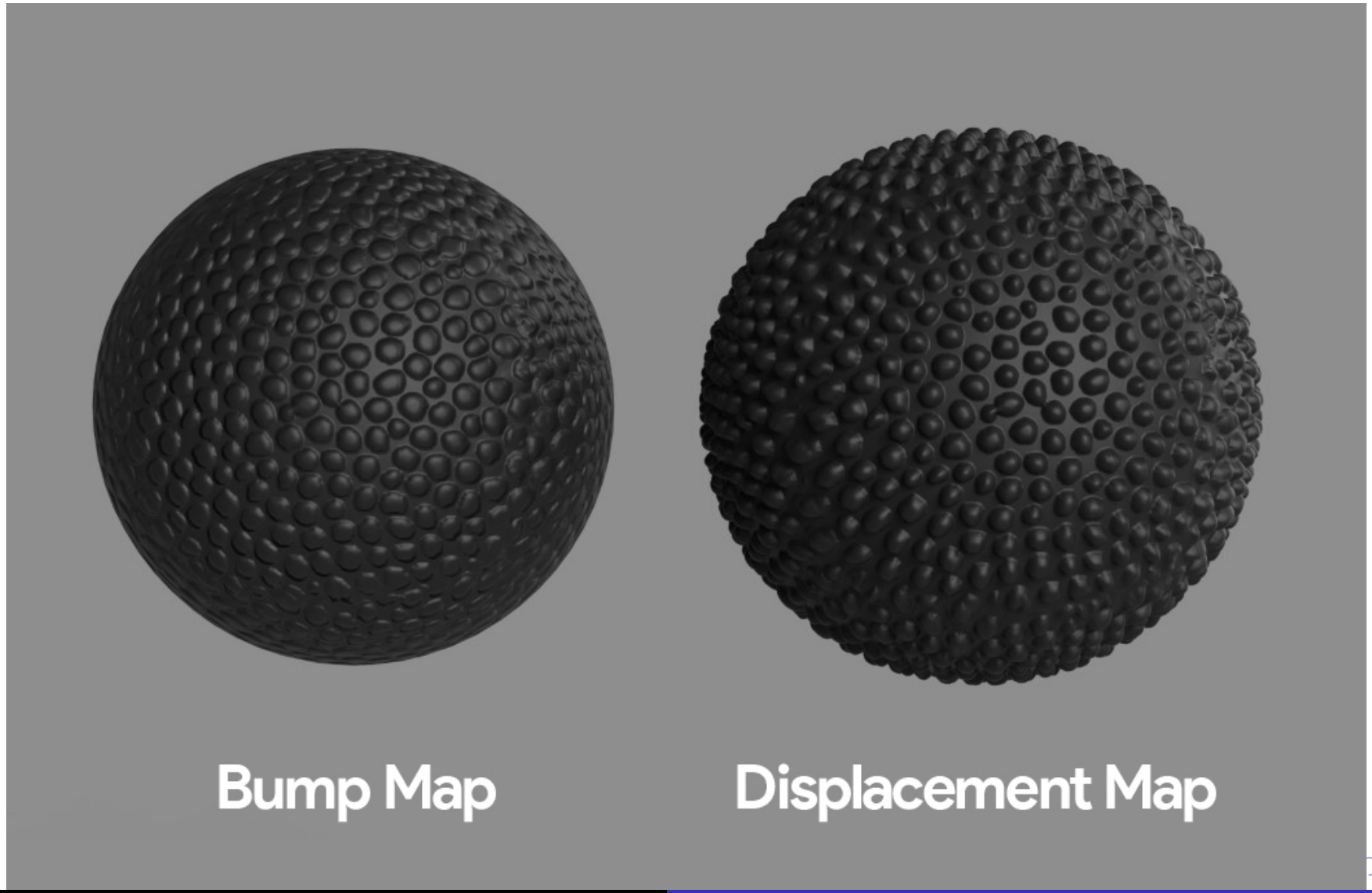
Section 11.4.3: Displacement Maps

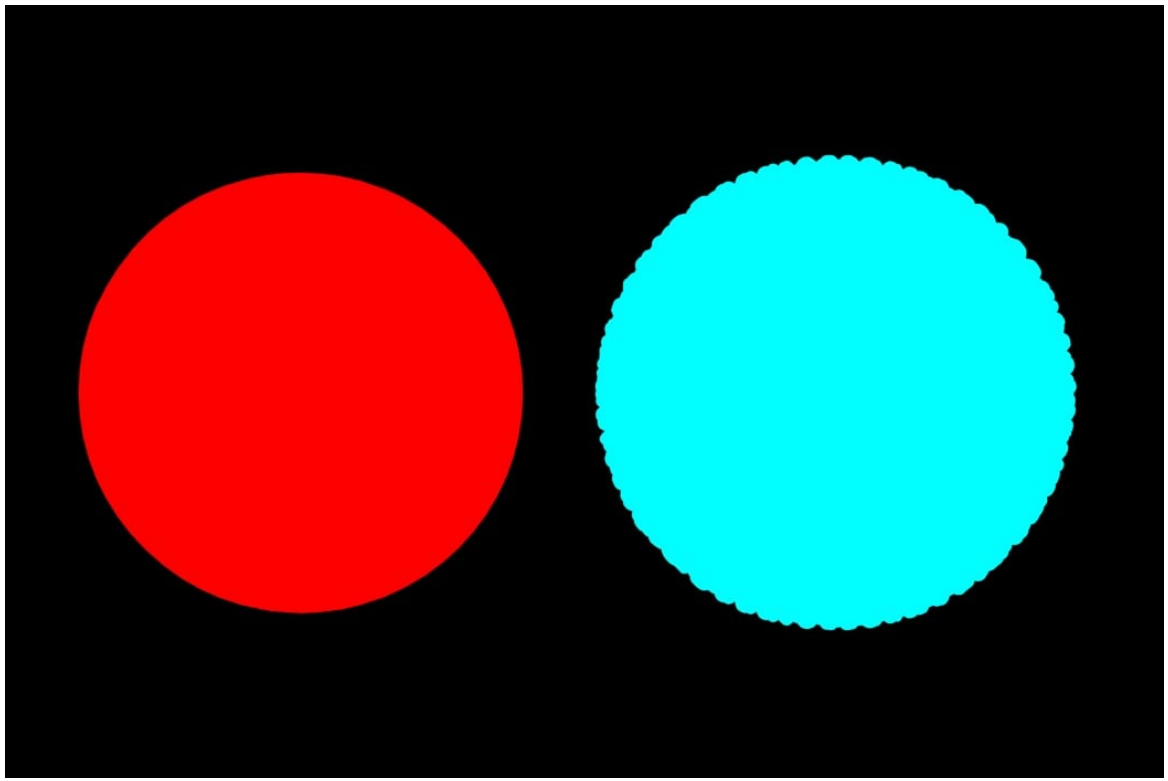
- Normal maps have a drawback, what is it?

Section 11.4.3: Displacement Maps

- Normal maps have a drawback, what is it?
  - They do not change the surface, rather just a shading trick.
- Displacement maps have the same idea as bump maps and are used to change the surface.
- Displacement maps are a scalar map that gives the height above the "average terrain"
- Displacement maps then move the surface along the normal to a new location given by the scalar.

`https://www.yankodesign.com/2019/04/13/creating-realistic-textures-with-displacement-maps-in-keys`

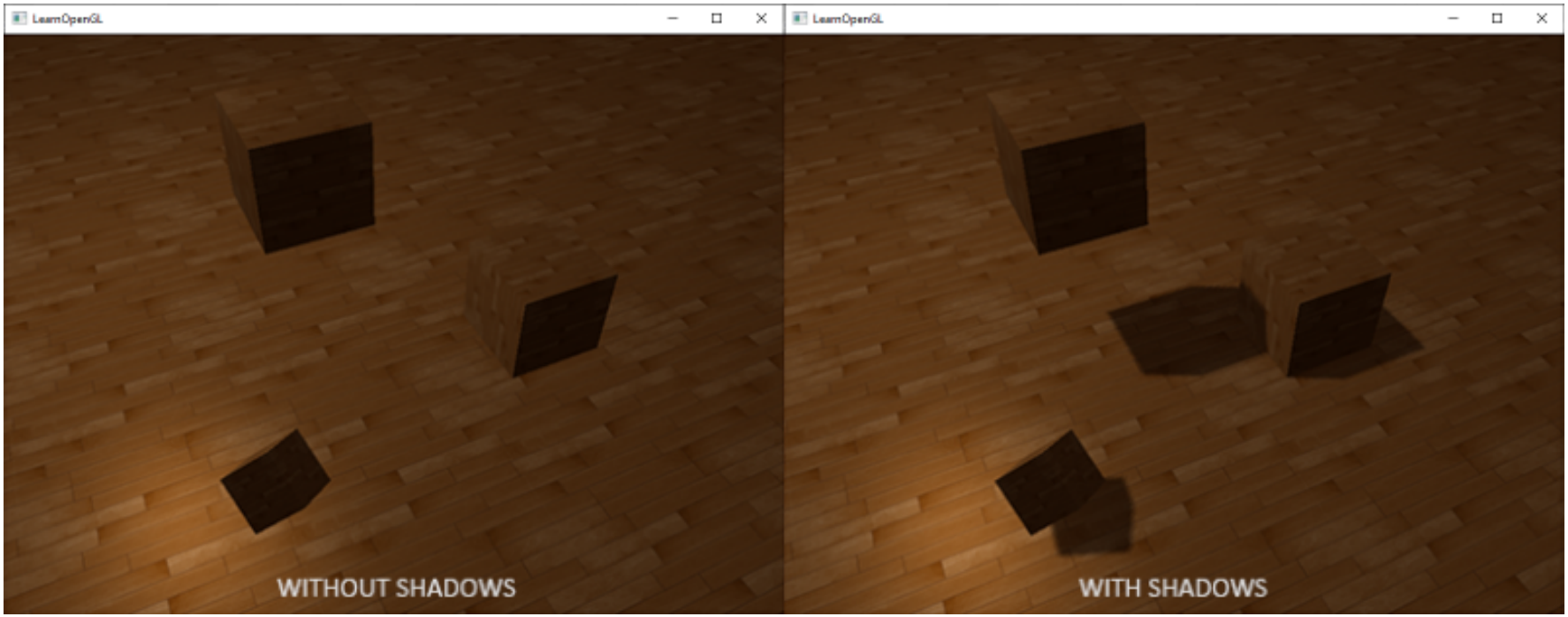

Bump Map
Displacement Map



Red is a bump map and cyan is a displacement map.

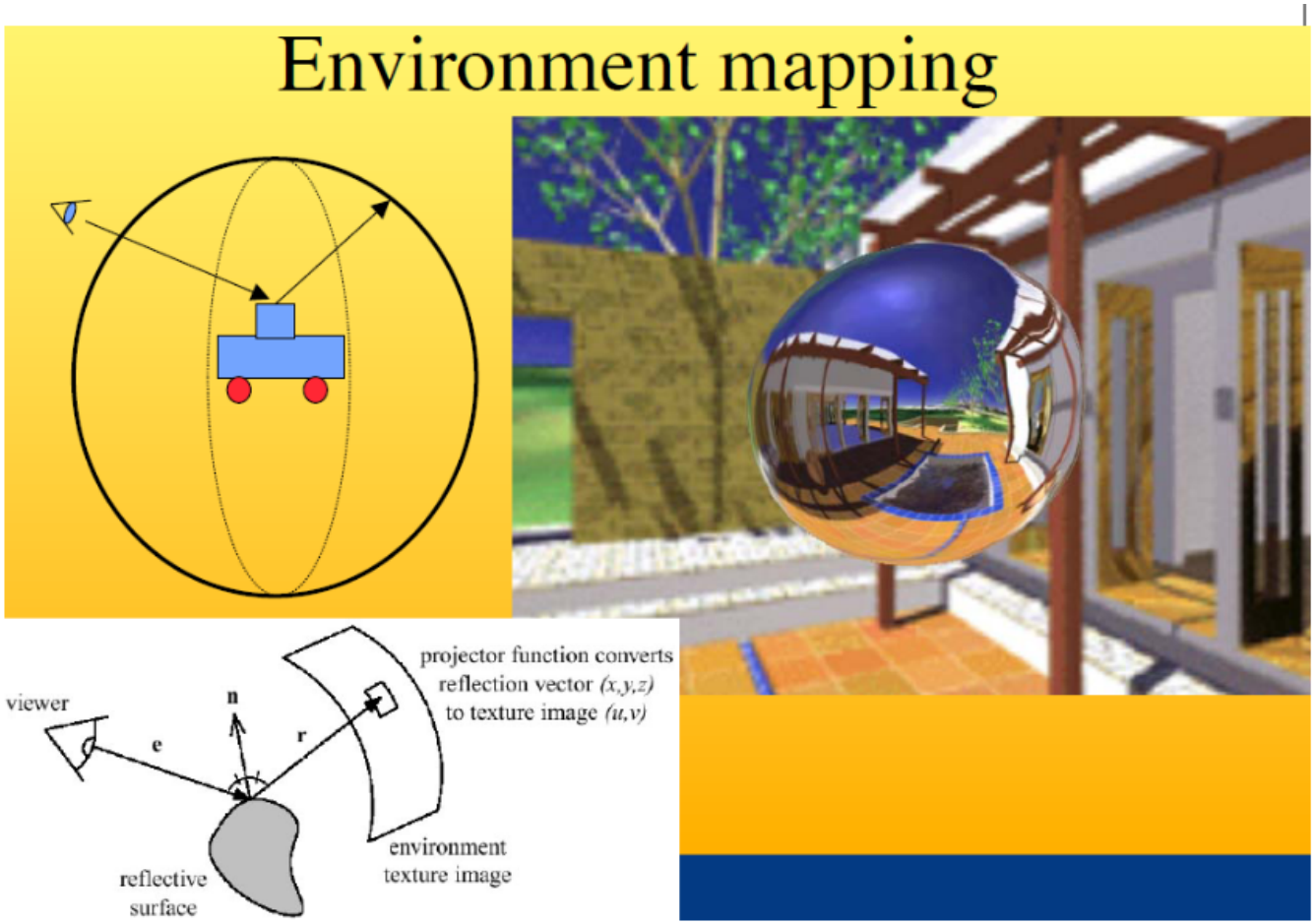
- Shadows are usually determined using a ray tracer.
- But this does not fit in so well with our rasterizer process.
- An alternative is to use a texture map for calculating the shadows of each surface.
- This is known as a shadow map.
- The idea behind shadow maps:
  - Represent the volume of space that are illuminated by a point light source.
  - *"we render the scene from the light's point of view and everything we see from the light's perspective is lit and everything we can't see must be in shadow"* - `https://learnopengl.com/Advanced-Lighting/Shadows/Shadow-Mapping`
- Think of a torch light.
  - The area that is illuminated is points that are inside the beam of light.

- Interestingly, this is the same volume that is visible with a perspective camera located at the same point as the light source.
- We thus can use a depth map that indicates how "deep" the object is in the volume.
  - For view volume, it is the z buffer
  - For lighting, it is the shadow map.
- The textbook goes into depth of how this can be implemented and might be useful for the homework assignment.



https://learnopengl.com/Advanced-Lighting/Shadows/Shadow-Mapping

This differs slightly from the textbook.



https://web.cs.wpi.edu/~emmanuel/courses/cs543/f13/slides/lecture09_p1.pdf

- Section 11.5 is left as it is tends a bit more towards the ideas of game development.
- Depending on time we might circle back to it.

- Reminder: you may use the shaders for the texture-related functionalities of Practical 4.
- All non-texture related requirements of the practical should still be done in the CPP part of the assignment, using practical 1 as the base. This includes but is not limited to lighting and transformations.
- To assist you, you are allowed to use the following library to assist with loading the texture image:
  - `std_image.h`
  - `https://raw.githubusercontent.com/nothings/stb/master/stb_image.h`

Note that this method follows the provided example. Alternative methods do exist:

- We will be creating the following render:

- To use the library, include the library as follows in the relevant file:

```
#define STB_IMAGE_IMPLEMENTATION
#include "stb_image.h"
```

- For each vertex in the surface that will be used to map the texture to, create a texture coordinate as well.
- For example:

```
float vertices[] = {
    -0.5f, -0.5f, // BL
    0.5f,  -0.5f, // BR
    0.5f,  0.5f,  // TR

    0.5f,  0.5f,  // TR
    -0.5f, 0.5f,  // TL
    -0.5f, -0.5f, // BL
};
```

```
float texture[] = {
    0.0f, 0.0f, // BL
    1.0f, 0.0f, // BR
    1.0f, 1.0f, // TR

    1.0f, 1.0f, // TR
    0.0f, 1.0f, // TL
    0.0f, 0.0f, // BL
};
```

Note the example also has a colour array, whose purpose is shown later.

Create a VBO for the positions, colours (if needed), and texture:

1.

```
GLuint texturebuffer;
glGenBuffers(1, &texturebuffer);
```

2.

```
glBindBuffer(GL_ARRAY_BUFFER, texturebuffer);
glBufferData(GL_ARRAY_BUFFER, sizeof(texture),
    texture, GL_STATIC_DRAW);
```

3.

```
glEnableVertexAttribArray(2);
glBindBuffer(GL_ARRAY_BUFFER, texturebuffer);
glVertexAttribPointer(2, 2, GL_FLOAT, GL_FALSE,
    0, (void *)0);
```

Now loading the image, named "wall.jpg":

```
int imgWidth, imgHeight, nrChannels;
unsigned char *data = stbi_load("wall.jpg",
    &imgWidth, &imgHeight, &nrChannels, 0);
if (!data)
{
    std::cerr << "Failed to load texture\n";
    return -1;
}
```

Bind the texture:

```
GLuint textureID;
glGenTextures(1, &textureID);
glBindTexture(GL_TEXTURE_2D, textureID);
```

- glGenTextures: https://docs.gl/gl3/glGenTextures
- glBindTexture: https://docs.gl/gl3/glBindTexture

Now configuring the parameters for how the texture is used:

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S,
    GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T,
    GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,
    GL_LINEAR);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER,
    GL_LINEAR);
```

- `glTexParameteri`: https://docs.gl/gl3/glTexParameter

Now determine the texture format:

```
GLenum format = (nrChannels == 4) ? GL_RGBA : GL_RGB;
glTexImage2D(GL_TEXTURE_2D, 0, format, imgWidth,
    imgHeight, 0, format, GL_UNSIGNED_BYTE, data);
glGenerateMipmap(GL_TEXTURE_2D);
```

- `glTexImage2D`: https://docs.gl/gl3/glTexImage2D

Free the image:

```
stbi_image_free(data);
```

Pass the texture to the shaders:

```
glUseProgram(programID);
glUniform1i(glGetUniformLocation(programID,
    "ourTexture"), 0);
```

In the rendering loop, activate the texture:

```
glActiveTexture(GL_TEXTURE0);
glBindTexture(GL_TEXTURE_2D, textureID);
```

- `glActiveTexture`: `https://docs.gl/gl3/glTexImage2D`

Now for our shaders:

- Vertex shader:

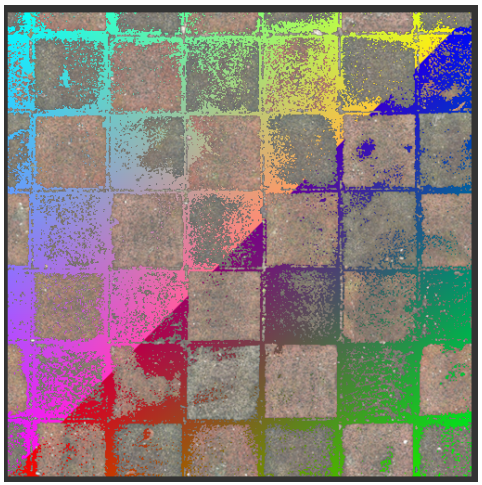
```
#version 330 core
layout(location = 0) in vec2 aPos;
layout(location = 1) in vec3 colour;
layout(location = 2) in vec2 aTexCoord;

out vec3 outColour;
out vec2 TexCoord;
out vec2 pos;

void main() {
    gl_Position = vec4(aPos, 0.0, 1.0);
    outColour = colour;
    TexCoord = aTexCoord;
    pos = aPos;
}
```

Now for our shaders:

- Fragment shader:

```
#version 330 core
out vec4 FragColor;

in vec3 outColour;
in vec2 TexCoord;
in vec2 pos;
uniform sampler2D ourTexture;

void main() {
    vec4 t = texture(ourTexture, TexCoord);
    FragColor = t;
}
```

```
#version 330 core
out vec4 FragColor;

in vec3 outColour;
in vec2 TexCoord;
in vec2 pos;
uniform sampler2D ourTexture;

void main() {
    vec4 t =
        texture(ourTexture,
        TexCoord);
    if(t.z < 0.4){
        FragColor =
            vec4(outColour,1.0);
    } else {
        FragColor = t;
    }
}
```



Joke of the day - By Claude

Why did the OpenGL texture coordinates go to couples therapy?

Joke of the day - By Claude

Why did the OpenGL texture coordinates go to couples therapy?

Because U and V just couldn't seem to wrap their relationship around the problem.